实验 2 报告

姓名：_____ 学号：_____ 班级：_____

实验二 DNS 与 HTTP 协议分析

实验目的

1）能够分析 DNS 协议的工作过程，

2）分析应用层协议（如 HTTP、HTTPS）的工作过程，理解应用层与传输层及下层协议的关系，分析缓存工作原理及好处。

3）能够依据 HTTP 协议编写简单的 HTTP 客户端程序，从几个网站去读取主页，或者取文件。

实验内容

1）利用现有校园网络及外网服务器搭建内网、外网环境；

2）用 Wireshark 截取 DNS 报文，分析 DNS 工作过程；

3）用 Wireshark 截获 HTTP/HTTPS 报文，分析报文结构及浏览器和服务器的交互过程；分析 HTTP 协议的缓存机制，分析 HTTPS 的加密传输机制。分析应用层协议跟 TCP/DNS 等协议的交互关系；

4）依据 HTTP 协议，编写简单的 HTTP 客户端程序，从几个网站去读取主页，或者取文件。

实验原理

略

实验环境与分组

依托校园网和外网服务器搭建内网、外网环境，每名同学单独完成实验并撰写实验报告。

实验步骤

2.3.1 DNS 协议分析

步骤 1：DNS 记录的本地缓存操作。

1.1 执行“ipconfig /displaydns”命令观察本地缓存，文字贴 2 条记录。

```
PS C:\Users\OMEN> ipconfig /displaydns

Windows IP 配置

    fe3cr.delivery.mp.microsoft.com
    ----------------------------------------
    记录名称. . . . . : fe3cr.delivery.mp.microsoft.com
    记录类型. . . . . : 5
    生存时间. . . . . : 515576
    数据长度. . . . . : 8
    部分. . . . . : 答案
    CNAME 记录  . . . . . : fe3.delivery.mp.microsoft.com


    记录名称. . . . . : fe3.delivery.mp.microsoft.com
    记录类型. . . . . : 5
    生存时间. . . . . : 515576
    数据长度. . . . . : 8
    部分. . . . . : 答案
    CNAME 记录  . . . . . : glb.cws.prod.dcat.dsp.trafficmanager.net


    记录名称. . . . . : glb.cws.prod.dcat.dsp.trafficmanager.net
    记录类型. . . . . : 1
    生存时间. . . . . : 515576
    数据长度. . . . . : 4
    部分. . . . . : 答案
    A (主机)记录  . . . . : 135.232.92.97


    i1.hdslb.com
    ----------------------------------------
    记录名称. . . . . : i1.hdslb.com
    记录类型. . . . . : 5
    生存时间. . . . . : 528866
    数据长度. . . . . : 8
    部分. . . . . : 答案
    CNAME 记录  . . . . . : i0.hdslb.com.cdn.dnsv1.com


    记录名称. . . . . : i0.hdslb.com.cdn.dnsv1.com
    记录类型. . . . . : 5
    生存时间. . . . . : 528866
    数据长度. . . . . : 8
    部分. . . . . : 答案
    CNAME 记录  . . . . . : 427884.sched.sma-dk.tdnsstic1.cn


    记录名称. . . . . : 427884.sched.sma-dk.tdnsstic1.cn
    记录类型. . . . . : 1
    生存时间. . . . . : 528866
    数据长度. . . . . : 4
    部分. . . . . : 答案
    A (主机)记录  . . . . : 111.51.105.102
```

1.2 执行“ipconfig /flushdns”清除本地缓存，然后再执行“ipconfig /displaydns”命令观察本地缓存，贴图。

```
PS C:\Users\OMEN> ipconfig /flushdns

Windows IP 配置

已成功刷新 DNS 解析缓存。
PS C:\Users\OMEN> ipconfig /displaydns

Windows IP 配置

    opcenter.example.com
    ----------------------------------------
    没有 AAAA 类型的记录

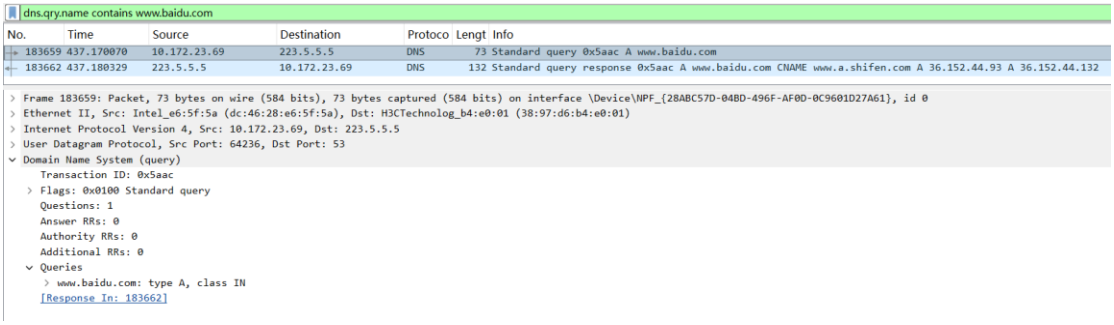

    记录名称. . . . . : opcenter.example.com
    记录类型. . . . . : 1
    生存时间. . . . . : 604800
    数据长度. . . . . : 4
    部分. . . . . : 答案
    A (主机)记录  . . . . : 202.117.21.118


    notebook.example.com
    ----------------------------------------
    没有 AAAA 类型的记录


    记录名称. . . . . : notebook.example.com
    记录类型. . . . . : 1
```

步骤 2：默认的 DNS 域名解析。运行 Wireshark 截获报文，浏览器访问域名（如 www.baidu.com），网站打开后停止报文截获。观察分析 DNS 查询、应答报文。

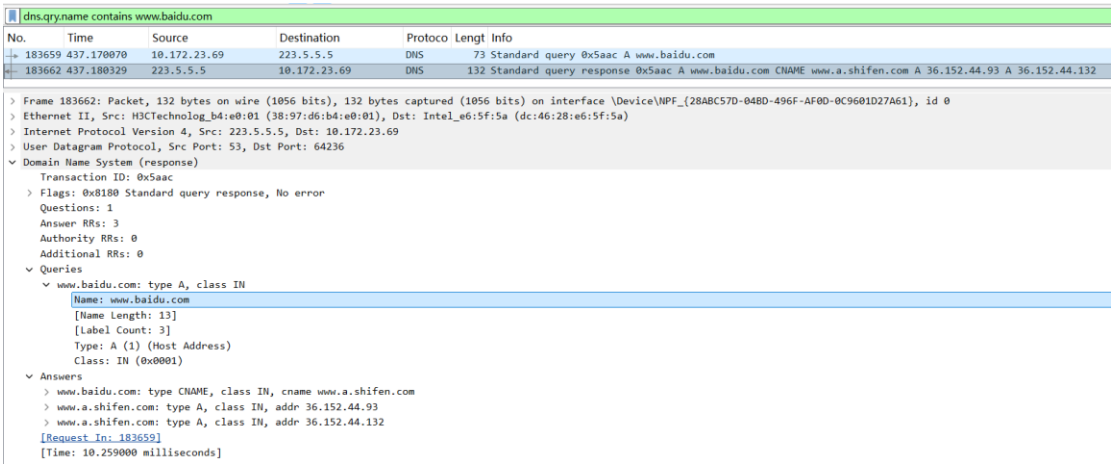
2.1 贴图（Wireshark 的“分组列表”窗口）。DNS 数据包过多时，可设置过滤条件：dns.qry.name contains www.baidu.com）。



2.2 分析 DNS 查询的报文，填表。

传输层协议（UDP 还是 TCP？）	**UDP**
目的地址	**223.5.5.5**
目的端口	**53**
域名解析记录类型	**A**
解析的名字	**www.baidu.com**

2.3 分析 DNS **应答**的报文，**贴图**给出该报文的“协议封装”。

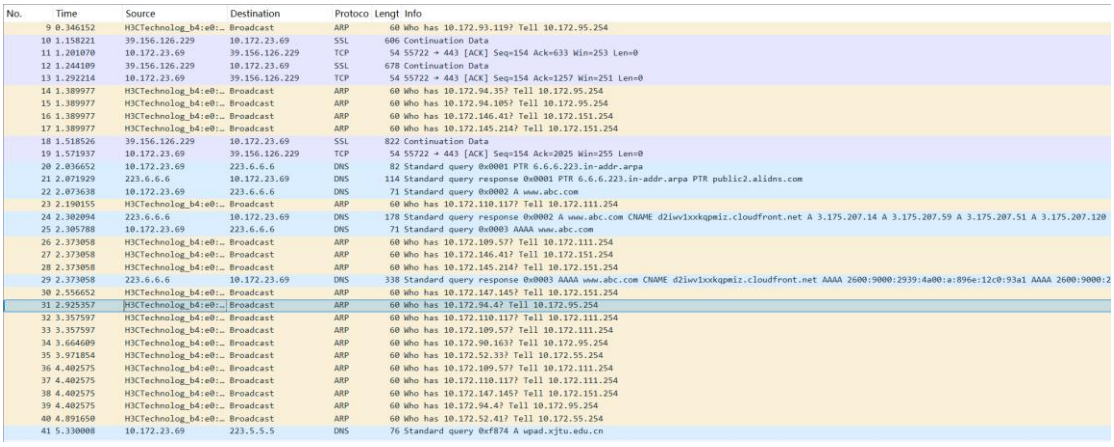


2.4 针对 2.3 中的报文分析填表。

有几条 Answers 记录	**3**
有几条 Authoritative NS 记录	**0**
有几条 Additional 记录	**0**

步骤 3：指定服务器的 DNS 域名解析。执行“ipconfig /flushdns”清除本地缓存；运行 Wireshark 截获报文，在命令窗口执行**指定 DNS 服务器**（如 **223.6.6.6**）解析域名 X（如 **www.abc.com**）的命令（nslookup **www.abc.com 223.6.6.6**），解析完毕后停止报文截获，观察分析 DNS 查询、回复报文分别包含哪些主要内容。

3.1 贴图（Wireshark 的“分组列表”窗口）。



3.2 说明相关的 DNS 报文有几条？

6 条

3.3 分析 DNS 报文，填表。

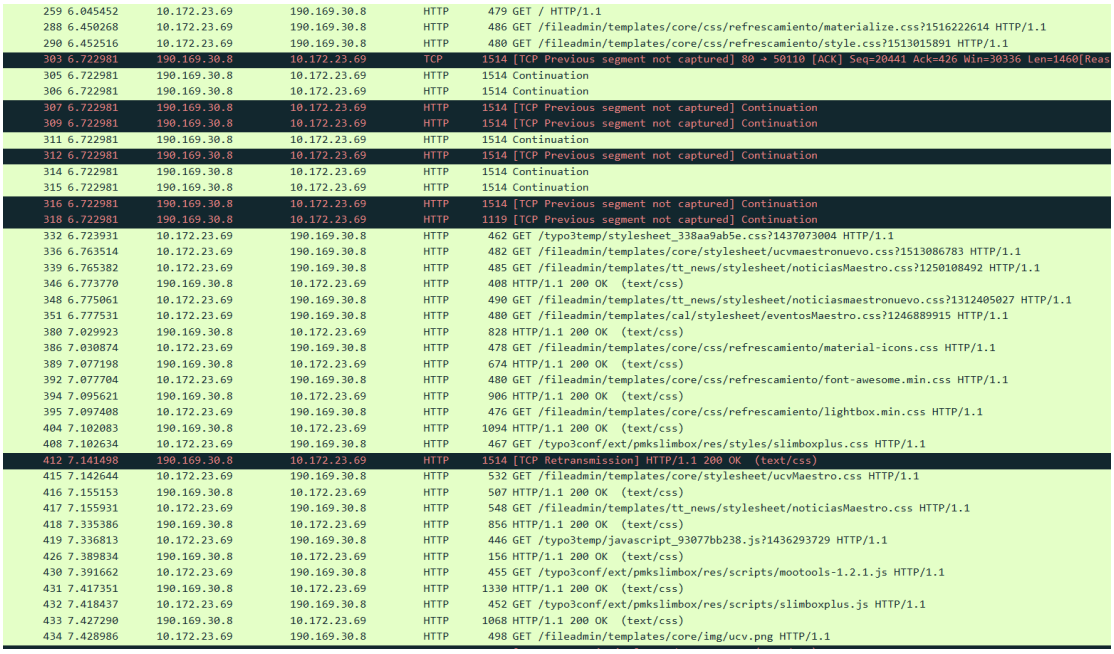
DNS 服务器地址与本机默认 DNS 服务器地址是否相同？	**否**
有没有 **PTR** 类型的域名解析报文？	**有**
www.abc.com 的真实名字是什么？	**d2iwv1xxkqpmiz.cloudfront.net**
其对应的 IPv4 地址有几个？	**4 个**

2.3.2 HTTP/HTTPS 协议分析

注意：步骤 4 和 5 牵涉到 DNS 和 HTTP 的缓存，请注意合理安排访问、抓包和分析的次序。可以先访问和抓包，然后慢慢分析。

步骤 4：无（DNS、HTTP）缓存情况下的**网站页面**访问速度测试。

4.1 清空 DNS 和 HTTP 浏览器的缓存，选择合适的网站（支持 HTTP），访问同时 Wireshark 抓包。对南半球网站的 HTTP 访问贴图（仅“分组列表”）。



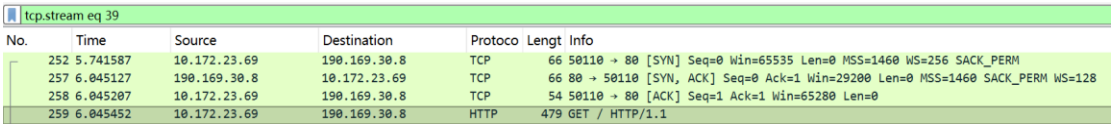
4.2 分析报文并填表。

无缓存情况下不同网站访问情况统计表

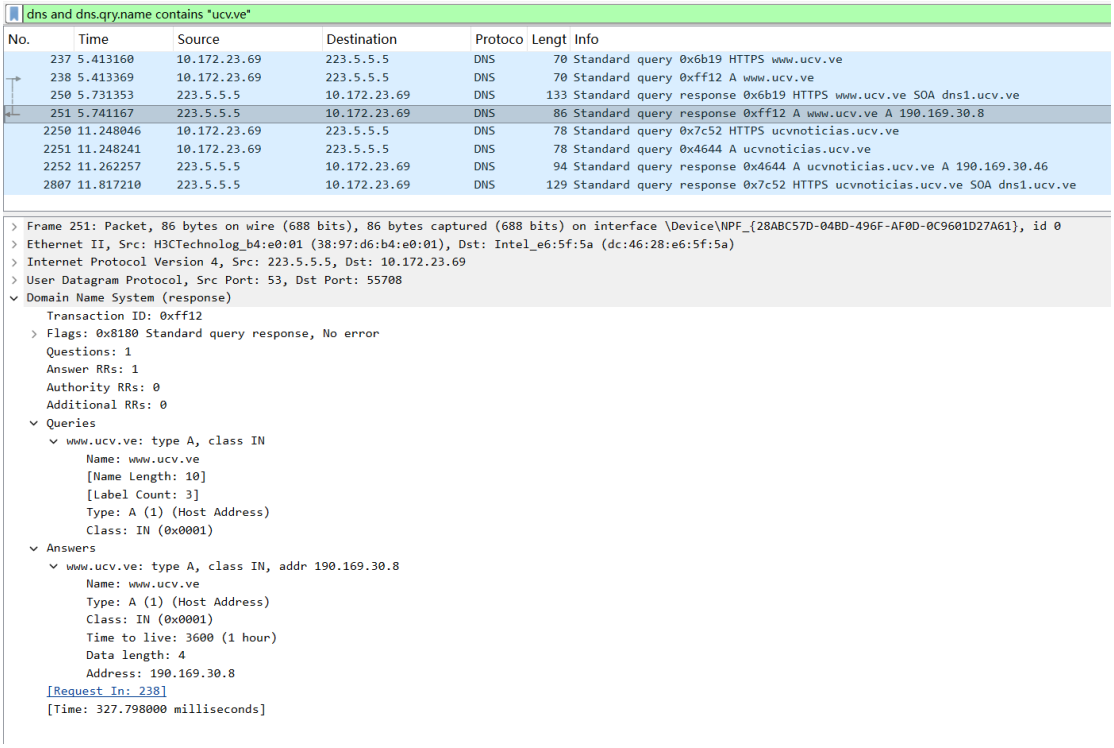
访问网站	国内	国外	南半球
域名	www.hit.edu.cn	www.customs.gov.mm	www.ucv.ve
IP 地址	202.118.254.135	150.95.82.190	190.169.30.8
RTT	41.263	561.954	303.54 ms
DNS 次数	8	8	4
第 1 个 DNS 请求的应答时间(ms)	8.467	665.077	318.193

页面的加载时间(ms)	14322.518	176388.699	19928.113
HTTP 建立连接数	7	16	6
请求对象数	114	47	124

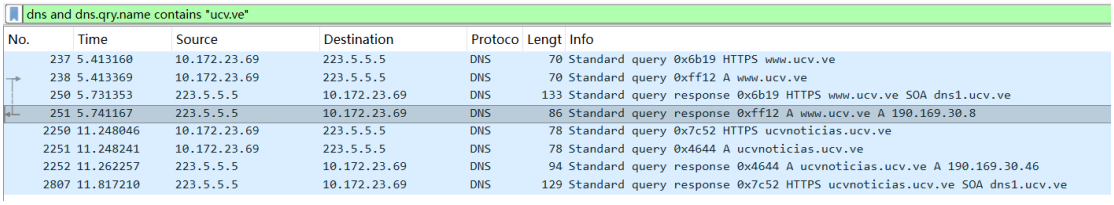
4.3 在南半球网站的 HTTP 访问中找到主页面下载时 TCP 连接的三次握手和 HTTP 请求报文，贴图（过滤器+“分组列表”）。

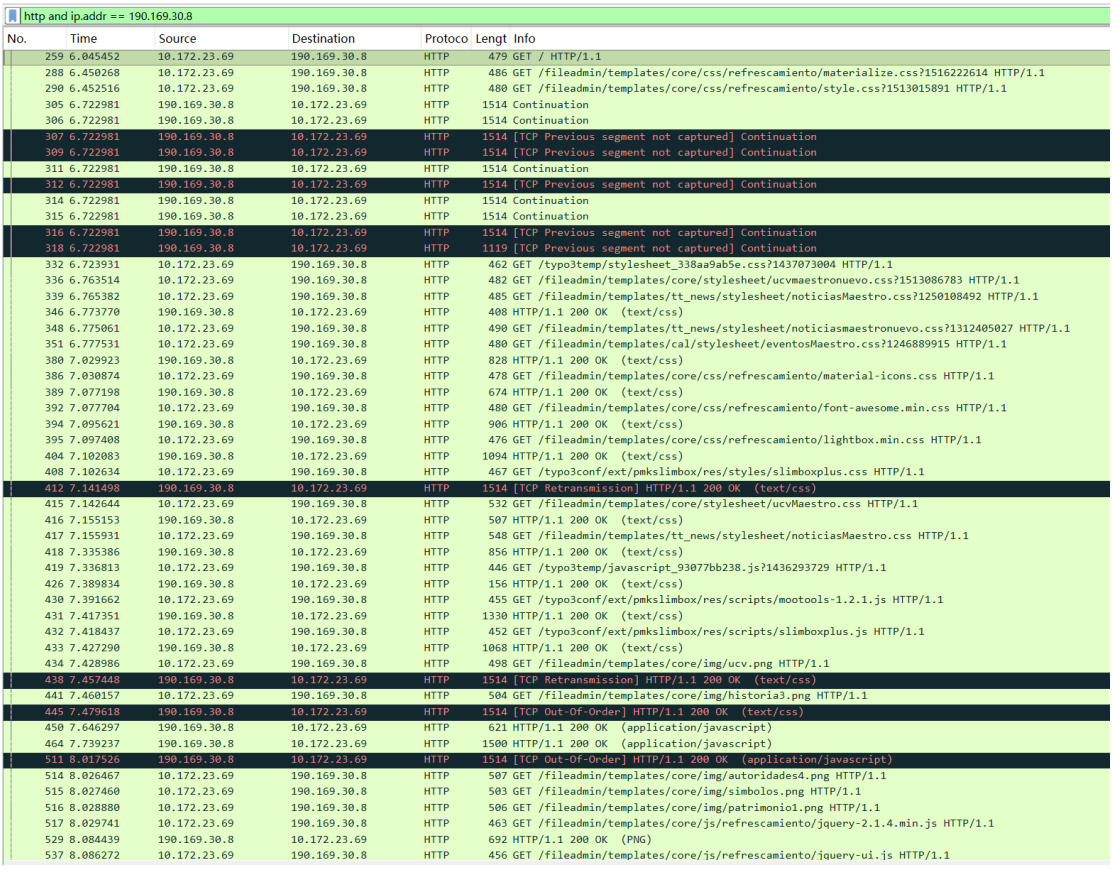


4.4 找到南半球网站的 DNS 解析请求和应答报文，贴图（过滤器+“分组列表”+应答报文的 DNS“协议”细节）。



4.5 找到访问南半球网站时牵涉到的 DNS 报文和 HTTP 报文。贴图（可以 2 个，包括过滤器+“分组列表”），并说明对应的“**页面的加载时间**(ms)”是如何得到的。





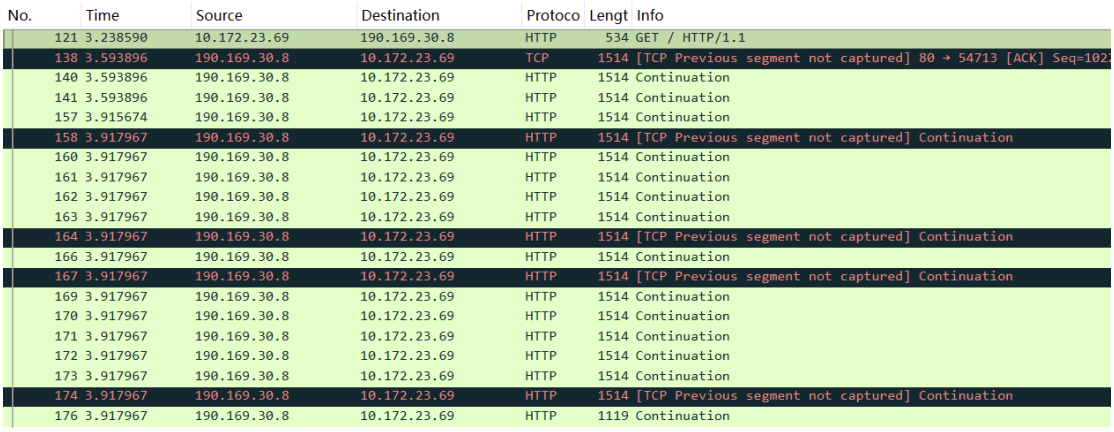
http and ip.addr == 190.169.30.8

No.	Time	Source	Destination	Protoco	Lengt	Info
259	6.045452	10.172.23.69	190.169.30.8	HTTP	479	GET / HTTP/1.1
288	6.450268	10.172.23.69	190.169.30.8	HTTP	486	GET /fileadmin/templates/core/css/refrescamiento/materialize.css?1516222614 HTTP/1.1
290	6.452516	10.172.23.69	190.169.30.8	HTTP	480	GET /fileadmin/templates/core/css/refrescamiento/style.css?1513015891 HTTP/1.1
305	6.722981	190.169.30.8	10.172.23.69	HTTP	1514	Continuation
306	6.722981	190.169.30.8	10.172.23.69	HTTP	1514	Continuation
307	6.722981	190.169.30.8	10.172.23.69	HTTP	1514	[TCP Previous segment not captured] Continuation
309	6.722981	190.169.30.8	10.172.23.69	HTTP	1514	[TCP Previous segment not captured] Continuation
311	6.722981	190.169.30.8	10.172.23.69	HTTP	1514	Continuation
312	6.722981	190.169.30.8	10.172.23.69	HTTP	1514	[TCP Previous segment not captured] Continuation
314	6.722981	190.169.30.8	10.172.23.69	HTTP	1514	Continuation
315	6.722981	190.169.30.8	10.172.23.69	HTTP	1514	Continuation
316	6.722981	190.169.30.8	10.172.23.69	HTTP	1514	[TCP Previous segment not captured] Continuation
318	6.722981	190.169.30.8	10.172.23.69	HTTP	1119	[TCP Previous segment not captured] Continuation
332	6.723931	10.172.23.69	190.169.30.8	HTTP	462	GET /typo3temp/stylesheet_338aa9ab5e.css?1437073004 HTTP/1.1
336	6.763514	10.172.23.69	190.169.30.8	HTTP	482	GET /fileadmin/templates/core/stylesheet/ucvmaestronuevo.css?1513086783 HTTP/1.1
339	6.765382	10.172.23.69	190.169.30.8	HTTP	485	GET /fileadmin/templates/tt_news/stylesheet/noticiasMaestro.css?1250108492 HTTP/1.1
346	6.773770	190.169.30.8	10.172.23.69	HTTP	408	HTTP/1.1 200 OK (text/css)
348	6.775061	10.172.23.69	190.169.30.8	HTTP	490	GET /fileadmin/templates/tt_news/stylesheet/noticiasmaestronuevo.css?1312405027 HTTP/1.1
351	6.777531	10.172.23.69	190.169.30.8	HTTP	480	GET /fileadmin/templates/cal/stylesheet/eventosMaestro.css?1246889915 HTTP/1.1
380	7.029923	190.169.30.8	10.172.23.69	HTTP	828	HTTP/1.1 200 OK (text/css)
386	7.030874	10.172.23.69	190.169.30.8	HTTP	478	GET /fileadmin/templates/core/css/refrescamiento/material-icons.css HTTP/1.1
389	7.077198	190.169.30.8	10.172.23.69	HTTP	674	HTTP/1.1 200 OK (text/css)
392	7.077704	10.172.23.69	190.169.30.8	HTTP	480	GET /fileadmin/templates/core/css/refrescamiento/font-awesome.min.css HTTP/1.1
394	7.095621	190.169.30.8	10.172.23.69	HTTP	906	HTTP/1.1 200 OK (text/css)
395	7.097408	10.172.23.69	190.169.30.8	HTTP	476	GET /fileadmin/templates/core/css/refrescamiento/lightbox.min.css HTTP/1.1
404	7.102083	190.169.30.8	10.172.23.69	HTTP	1094	HTTP/1.1 200 OK (text/css)
408	7.102634	10.172.23.69	190.169.30.8	HTTP	467	GET /typo3conf/ext/pmkslimbox/res/styles/slimboxplus.css HTTP/1.1
412	7.141498	190.169.30.8	10.172.23.69	HTTP	1514	[TCP Retransmission] HTTP/1.1 200 OK (text/css)
415	7.142644	10.172.23.69	190.169.30.8	HTTP	532	GET /fileadmin/templates/core/stylesheet/ucvMaestro.css HTTP/1.1
416	7.155153	190.169.30.8	10.172.23.69	HTTP	507	HTTP/1.1 200 OK (text/css)
417	7.155931	10.172.23.69	190.169.30.8	HTTP	548	GET /fileadmin/templates/tt_news/stylesheet/noticiasMaestro.css HTTP/1.1
418	7.335386	190.169.30.8	10.172.23.69	HTTP	856	HTTP/1.1 200 OK (text/css)
419	7.336813	10.172.23.69	190.169.30.8	HTTP	446	GET /typo3temp/javascript_93077bb238.js?1436293729 HTTP/1.1
426	7.389834	190.169.30.8	10.172.23.69	HTTP	156	HTTP/1.1 200 OK (text/css)
430	7.391662	10.172.23.69	190.169.30.8	HTTP	455	GET /typo3conf/ext/pmkslimbox/res/scripts/mootools-1.2.1.js HTTP/1.1
431	7.417351	190.169.30.8	10.172.23.69	HTTP	1330	HTTP/1.1 200 OK (text/css)
432	7.418437	10.172.23.69	190.169.30.8	HTTP	452	GET /typo3conf/ext/pmkslimbox/res/scripts/slimboxplus.js HTTP/1.1
433	7.427290	190.169.30.8	10.172.23.69	HTTP	1068	HTTP/1.1 200 OK (text/css)
434	7.428986	10.172.23.69	190.169.30.8	HTTP	498	GET /fileadmin/templates/core/img/ucv.png HTTP/1.1
438	7.457448	190.169.30.8	10.172.23.69	HTTP	1514	[TCP Retransmission] HTTP/1.1 200 OK (text/css)
441	7.460157	10.172.23.69	190.169.30.8	HTTP	504	GET /fileadmin/templates/core/img/historia3.png HTTP/1.1
445	7.479618	190.169.30.8	10.172.23.69	HTTP	1514	[TCP Out-Of-Order] HTTP/1.1 200 OK (text/css)
450	7.646297	190.169.30.8	10.172.23.69	HTTP	621	HTTP/1.1 200 OK (application/javascript)
464	7.739237	190.169.30.8	10.172.23.69	HTTP	1500	HTTP/1.1 200 OK (application/javascript)
511	8.017526	190.169.30.8	10.172.23.69	HTTP	1514	[TCP Out-Of-Order] HTTP/1.1 200 OK (application/javascript)
514	8.026467	10.172.23.69	190.169.30.8	HTTP	507	GET /fileadmin/templates/core/img/autoridades4.png HTTP/1.1
515	8.027460	10.172.23.69	190.169.30.8	HTTP	503	GET /fileadmin/templates/core/img/simbolos.png HTTP/1.1
516	8.028880	10.172.23.69	190.169.30.8	HTTP	506	GET /fileadmin/templates/core/img/patrimonio1.png HTTP/1.1
517	8.029741	10.172.23.69	190.169.30.8	HTTP	463	GET /fileadmin/templates/core/js/refrescamiento/jquery-2.1.4.min.js HTTP/1.1
529	8.084439	190.169.30.8	10.172.23.69	HTTP	692	HTTP/1.1 200 OK (PNG)
537	8.086272	10.172.23.69	190.169.30.8	HTTP	456	GET /fileadmin/templates/core/js/refrescamiento/jquery-ui.js HTTP/1.1

页面的加载时间：从第一个相关 DNS 请求发出时刻，到最后一个相关 HTTP 响应报文到达（即最后一个对象响应完毕）时刻的时间差。

步骤 5：有（DNS、HTTP）缓存情况下的**网站页面**访问速度测试。

5.1 在访问一个网站后 2-3 分钟（或合适的时间）内再次访问该网站。同时 Wireshark 抓包。对南半球网站的 HTTP 访问贴图（仅“分组列表”）。



No.	Time	Source	Destination	Protoco	Lengt	Info
121	3.238590	10.172.23.69	190.169.30.8	HTTP	534	GET / HTTP/1.1
138	3.593896	190.169.30.8	10.172.23.69	TCP	1514	[TCP Previous segment not captured] 80 → 54713 [ACK] Seq=102
140	3.593896	190.169.30.8	10.172.23.69	HTTP	1514	Continuation
141	3.593896	190.169.30.8	10.172.23.69	HTTP	1514	Continuation
157	3.915674	190.169.30.8	10.172.23.69	HTTP	1514	Continuation
158	3.917967	190.169.30.8	10.172.23.69	HTTP	1514	[TCP Previous segment not captured] Continuation
160	3.917967	190.169.30.8	10.172.23.69	HTTP	1514	Continuation
161	3.917967	190.169.30.8	10.172.23.69	HTTP	1514	Continuation
162	3.917967	190.169.30.8	10.172.23.69	HTTP	1514	Continuation
163	3.917967	190.169.30.8	10.172.23.69	HTTP	1514	Continuation
164	3.917967	190.169.30.8	10.172.23.69	HTTP	1514	[TCP Previous segment not captured] Continuation
166	3.917967	190.169.30.8	10.172.23.69	HTTP	1514	Continuation
167	3.917967	190.169.30.8	10.172.23.69	HTTP	1514	[TCP Previous segment not captured] Continuation
169	3.917967	190.169.30.8	10.172.23.69	HTTP	1514	Continuation
170	3.917967	190.169.30.8	10.172.23.69	HTTP	1514	Continuation
171	3.917967	190.169.30.8	10.172.23.69	HTTP	1514	Continuation
172	3.917967	190.169.30.8	10.172.23.69	HTTP	1514	Continuation
173	3.917967	190.169.30.8	10.172.23.69	HTTP	1514	Continuation
174	3.917967	190.169.30.8	10.172.23.69	HTTP	1514	[TCP Previous segment not captured] Continuation
176	3.917967	190.169.30.8	10.172.23.69	HTTP	1119	Continuation

5.2 分析报文并填表。

带缓存情况下不同网站访问情况统计表

访问网站	国内	国外	南半球
域名	www.hit.edu.cn	www.customs.gov.mm	www.ucv.ve
IP 地址	202.118.254.135	150.95.82.190	190.169.30.8
RTT	41.897	558.214	310.042
DNS 次数	0	0	0
页面的加载时间 (ms)	773.014	12486.371	679.377
请求对象数	6	3	1
TCP 连接数	1	6	1

5.3 根据所学知识分析缓存技术会给页面访问带来哪些方面的改善，这些改善在上面的步骤 4、5 中有没有实际发生？请结合 4.2 和 5.2 两个表格数据说一说。

① 减少 DNS 解析开销：如果域名解析结果已经缓存在浏览器或操作系统中，再次访问同一网站时就不需要重新向 DNS 服务器发请求，因此 DNS 查询次数会减少，甚至变成 0。这样不仅减少了额外报文，也省去了等待 DNS 应答的时间。

② 减少页面资源的重复获取：网页中的 HTML、图片、CSS、JS 等对象，如果已经缓存在本地，再次访问时浏览器可能直接使用本地副本，或者只进行少量校验，而不用像第一次那样重新下载全部资源。因此，请求对象数会减少，TCP 连接数也可能减少。

③ 缩短页面加载时间：由于少了 DNS 查询、资源下载、连接建立等过程，页面整体完成加载所需的时间通常会明显下降。这是缓存带来的最直观改善。

④ 减轻服务器和网络负担：缓存命中后，客户端发出的请求更少，网络上传输的数据更少，服务器处理的请求也更少。

步骤 6：分析 HTTP 的缓存工作原理。

6.1 从前面步骤中找到 HTTP 应答包中**状态码为 304** 的数据包（过滤条件 "http.response.code == 304"）。**贴图**（仅“分组列表”）。

No.	Time	Source	Destination	Protoco	Lengt	Info
18775	4.741094	36.163.195.153	10.172.23.69	HTTP	356	HTTP/1.1 304 Not Modified
24526	5.931485	36.163.195.153	10.172.23.69	HTTP	356	HTTP/1.1 304 Not Modified
24826	6.754647	36.163.195.153	10.172.23.69	HTTP	356	HTTP/1.1 304 Not Modified
24921	7.510380	36.163.195.153	10.172.23.69	HTTP	356	HTTP/1.1 304 Not Modified
25026	8.631454	36.163.195.153	10.172.23.69	HTTP	356	HTTP/1.1 304 Not Modified

6.2 找到某 304 应答包及其对应的**请求包**，对请求报文**贴图**，注意展开该报文 HTTP 协议。

```
> Frame 18680: Packet, 668 bytes on wire (5344 bits), 668 bytes captured (5344 bits) on interface \Device\NPF_{28ABC57D-04BD-496F-AF0D-0C9601D27A61}, id 0
> Ethernet II, Src: Intel_e6:5f:5a (dc:46:28:e6:5f:5a), Dst: H3CTechnolog_b4:e0:01 (38:97:d6:b4:e0:01)
> Internet Protocol Version 4, Src: 10.172.23.69, Dst: 36.163.195.153
> Transmission Control Protocol, Src Port: 49666, Dst Port: 80, Seq: 15604, Ack: 1498263, Len: 614
v Hypertext Transfer Protocol
  > GET / HTTP/1.1\r\n
    Host: www.people.com.cn\r\n
    Connection: keep-alive\r\n
    Cache-Control: max-age=0\r\n
    Upgrade-Insecure-Requests: 1\r\n
    User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 Edg/146.0.0.0\r\n
    Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7\r\n
    Accept-Encoding: gzip, deflate\r\n
    Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6\r\n
  > Cookie: sso_c=0; sfr=1\r\n
    If-None-Match: W/"69ba9d5f-25df5"\r\n
    If-Modified-Since: Wed, 18 Mar 2026 12:41:03 GMT\r\n
    \r\n
    [Response in frame: 18775]
    [Full request URI: http://www.people.com.cn/]
```

6.3 对 304 应答包**贴图**，注意展开该报文 HTTP 协议。

```
> Frame 18775: Packet, 356 bytes on wire (2848 bits), 356 bytes captured (2848 bits) on interface \Device\NPF_{28ABC57D-04BD-496F-AF0D-0C9601D27A61}, id 0
> Ethernet II, Src: H3CTechnolog_b4:e0:01 (38:97:d6:b4:e0:01), Dst: Intel_e6:5f:5a (dc:46:28:e6:5f:5a)
> Internet Protocol Version 4, Src: 36.163.195.153, Dst: 10.172.23.69
> Transmission Control Protocol, Src Port: 80, Dst Port: 49666, Seq: 1498263, Ack: 16218, Len: 302
v Hypertext Transfer Protocol
  > HTTP/1.1 304 Not Modified\r\n
    Date: Wed, 18 Mar 2026 13:06:59 GMT\r\n
    Content-Type: text/html\r\n
    Connection: keep-alive\r\n
    Last-Modified: Wed, 18 Mar 2026 12:41:03 GMT\r\n
    ETag: "69ba9d5f-25df5"\r\n
    Age: 29\r\n
    x-ws-request-id: 69baa373_PS-XIY-016qy38_63841-43370\r\n
    x-via: 1.1 PS-XIY-016qy38:33 (Cdn Cache Server V2.0)\r\n
    \r\n
    [Request in frame: 18680]
    [Time since request: 18.962000 milliseconds]
    [Request URI: /]
    [Full request URI: http://www.people.com.cn/]
```

步骤 7：HTTP 流的跟踪。选中一条 HTTP 报文，点击右键，选择“追踪流->HTTP Stream”。贴图。



步骤 8：使用 ncat 工具访问某 HTTP 网站，把 ncat 命令及其输出用文字贴在框内（内容特别长时可截取到 HTTP 应答和 HTML 的前几行即可）。

```
C:\Users\OMEN\Desktop\文件\课程\计网实验\nmap-7.91\nmap-7.91>ncat -C www.xjtu.edu.cn 80
GET / HTTP/1.1
Host: www.xjtu.edu.cn
User-Agent: Zhang Zhaoyuan
Connection: close

HTTP/1.1 200 OK
Date: Wed, 18 Mar 2026 13:24:55 GMT
Server: **********
X-Frame-Options: SAMEORIGIN
Upgrade: h2c,h2
Connection: Upgrade, close
Last-Modified: Wed, 18 Mar 2026 12:28:05 GMT
Accept-Ranges: bytes
Content-Length: 57957
Cache-Control: max-age=600
Expires: Wed, 18 Mar 2026 13:34:55 GMT
Vary: Accept-Encoding
ETag: "e265-64d4b957a88ff-gzip"
Content-Type: text/html
Content-Language: zh-CN

锘?!DOCTYPE HTML>
<HTML><HEAD><TITLE>瑗垮附浜ら€氫ぇ瀛?/TITLE><META Name="keywords" Content="瑗垮附浜ら€氫ぇ
瀛? />

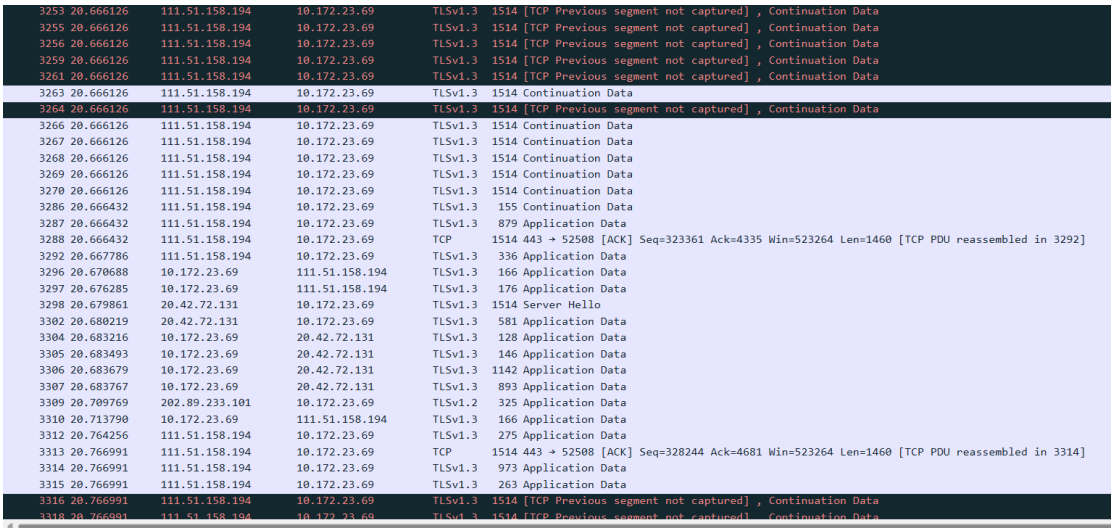



<META name="360-site-verification" content="845cb73defc117caad1186ca8fac8532"><script type="text/javascript">

if(/AppleWebKit.*Mobile/i.test(navigator.userAgent)
||(/MIDP|SymbianOS|NOKIA|SAMSUNG|LG|NEC|TCL|Alcatel|BIRD|DBTEL|Dopod|PHILIPS|HAIER|LENOVO|M
OT-|Nokia|SonyEricsson|SIE-|Amoi|ZTE/.test(navigator.userAgent))){
    if(window.location.href.indexOf("?mobile")<0){
        try{
            if(/Android|Windows Phone|webOS|iPhone|iPod|BlackBerry/i.test(navigator.userAgent)){
                window.location.href="http://mob.xjtu.edu.cn/";
            }
        }catch(e){}
    }
}

</script>
<META content="text/html; charset=UTF-8" http-equiv="Content-Type">
<META content="IE=edge,chrome=1" http-equiv="X-UA-Compatible"><LINK rel="stylesheet" type="text/css"
href="style/xjnew611.css"><script type="text/javascript" src="js/jquery.min.js"></script><script type="text/javascript"
src="js/jquery.SuperSlide.js"></script>
<script type="text/javascript" src="js/ddsmoothmenu.js"></script>
<META name="baidu-site-verification" content="r23jsHKdp1">
<STYLE>
.menulink ul li:hover ul{width: 230px !important;}
</STYLE>
```

步骤 9：HTTPS 协议分析。访问 HTTPS 网站时，因数据是加密传输无法观察内容，可结合原理图与抓到的数据包**分析 TLS 加密传输的建立过程、传输端口**等。贴图（仅给出某 TLS 的分组列表即可）



3253	20.666126	111.51.158.194	10.172.23.69	TLSv1.3	1514 [TCP Previous segment not captured] , Continuation Data
3255	20.666126	111.51.158.194	10.172.23.69	TLSv1.3	1514 [TCP Previous segment not captured] , Continuation Data
3256	20.666126	111.51.158.194	10.172.23.69	TLSv1.3	1514 [TCP Previous segment not captured] , Continuation Data
3259	20.666126	111.51.158.194	10.172.23.69	TLSv1.3	1514 [TCP Previous segment not captured] , Continuation Data
3261	20.666126	111.51.158.194	10.172.23.69	TLSv1.3	1514 [TCP Previous segment not captured] , Continuation Data
3263	20.666126	111.51.158.194	10.172.23.69	TLSv1.3	1514 Continuation Data
3264	20.666126	111.51.158.194	10.172.23.69	TLSv1.3	1514 [TCP Previous segment not captured] , Continuation Data
3266	20.666126	111.51.158.194	10.172.23.69	TLSv1.3	1514 Continuation Data
3267	20.666126	111.51.158.194	10.172.23.69	TLSv1.3	1514 Continuation Data
3268	20.666126	111.51.158.194	10.172.23.69	TLSv1.3	1514 Continuation Data
3269	20.666126	111.51.158.194	10.172.23.69	TLSv1.3	1514 Continuation Data
3270	20.666126	111.51.158.194	10.172.23.69	TLSv1.3	1514 Continuation Data
3286	20.666432	111.51.158.194	10.172.23.69	TLSv1.3	155 Continuation Data
3287	20.666432	111.51.158.194	10.172.23.69	TLSv1.3	879 Application Data
3288	20.666432	111.51.158.194	10.172.23.69	TCP	1514 443 → 52508 [ACK] Seq=323361 Ack=4335 Win=523264 Len=1460 [TCP PDU reassembled in 3292]
3292	20.667786	111.51.158.194	10.172.23.69	TLSv1.3	336 Application Data
3296	20.670688	10.172.23.69	111.51.158.194	TLSv1.3	166 Application Data
3297	20.676285	10.172.23.69	111.51.158.194	TLSv1.3	176 Application Data
3298	20.679861	20.42.72.131	10.172.23.69	TLSv1.3	1514 Server Hello
3302	20.680219	20.42.72.131	10.172.23.69	TLSv1.3	581 Application Data
3304	20.683216	10.172.23.69	20.42.72.131	TLSv1.3	128 Application Data
3305	20.683493	10.172.23.69	20.42.72.131	TLSv1.3	146 Application Data
3306	20.683679	10.172.23.69	20.42.72.131	TLSv1.3	1142 Application Data
3307	20.683767	10.172.23.69	20.42.72.131	TLSv1.3	893 Application Data
3309	20.709769	202.89.233.101	10.172.23.69	TLSv1.2	325 Application Data
3310	20.713790	10.172.23.69	111.51.158.194	TLSv1.3	166 Application Data
3312	20.764256	111.51.158.194	10.172.23.69	TLSv1.3	275 Application Data
3313	20.766991	111.51.158.194	10.172.23.69	TCP	1514 443 → 52508 [ACK] Seq=328244 Ack=4681 Win=523264 Len=1460 [TCP PDU reassembled in 3314]
3314	20.766991	111.51.158.194	10.172.23.69	TLSv1.3	973 Application Data
3315	20.766991	111.51.158.194	10.172.23.69	TLSv1.3	263 Application Data
3316	20.766991	111.51.158.194	10.172.23.69	TLSv1.3	1514 [TCP Previous segment not captured] , Continuation Data
3318	20.766991	111.51.158.194	10.172.23.69	TLSv1.3	1514 [TCP Previous segment not captured] , Continuation Data

2.3.3 编写 HTTP 客户端程序

步骤 10：按指导书要求编写 HTTP 客户端程序代码，并进行测试。记录 DNS 和 TCP 连接及获得 HTTP 完整应答的时间。

10.1 给出完整的代码（文字）。

```
import socket
import sys
import time

host = sys.argv[1]
port = 80

# DNS 解析计时
dns_start = time.perf_counter()
addr_info = socket.getaddrinfo(host, port, socket.AF_INET, socket.SOCK_STREAM)
dns_end = time.perf_counter()

ip = addr_info[0][4][0]

# 创建 TCP 连接并计时
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

tcp_start = time.perf_counter()
sock.connect((ip, port))
tcp_end = time.perf_counter()

# 手动输入请求头
lines = []
while True:
    line = input()
    if line == "":
        break
    lines.append(line)

request = "\r\n".join(lines) + "\r\n\r\n"

# 发送请求并接收完整应答计时
http_start = time.perf_counter()
sock.sendall(request.encode())
```

```
response = b""
while True:
    data = sock.recv(4096)
    if not data:
        break
    response += data
http_end = time.perf_counter()

sock.close()

print(f"IP: {ip}")
print(f"DNS time: {(dns_end - dns_start) * 1000:.3f} ms")
print(f"TCP connect time: {(tcp_end - tcp_start) * 1000:.3f} ms")
print(f"HTTP response time: {(http_end - http_start) * 1000:.3f} ms")
print()
print(response.decode(errors="ignore"))
```

10.2 给出测试结果（贴图）。

```
PS C:\Users\OMEN\Desktop\文件\课程\计网实验\实验2> python http_client.py www.xjtu.edu.cn
GET / HTTP/1.1
Host: www.xjtu.edu.cn
User-Agent: Zhang Zhaoyuan
Connection: close

IP: 202.117.1.13
DNS time: 10.734 ms
TCP connect time: 4.819 ms
HTTP response time: 14.004 ms

HTTP/1.1 200 OK
Date: Wed, 18 Mar 2026 13:35:19 GMT
Server: **********
X-Frame-Options: SAMEORIGIN
Upgrade: h2c,h2
Connection: Upgrade, close
Last-Modified: Wed, 18 Mar 2026 12:28:05 GMT
Accept-Ranges: bytes
Content-Length: 57957
Cache-Control: max-age=600
Expires: Wed, 18 Mar 2026 13:45:19 GMT
Vary: Accept-Encoding
ETag: "e265-64d4b957a88ff-gzip"
Content-Type: text/html
Content-Language: zh-CN

 <!DOCTYPE HTML>
<HTML><HEAD><TITLE>西安交通大学</TITLE><META Name="keywords" Content="西安交通大学" />




<META name="360-site-verification" content="845cb73defc117caad1186ca8fac8532"><script type="text/javascript">

if(/AppleWebKit.*Mobile/i.test(navigator.userAgent) ||(/MIDP|SymbianOS|NOKIA|SAMSUNG|LG|NEC|TCL|Alcatel|BIRD|DBTEL|Dopod |
|PHILIPS|HAIER|LENOVO|MOT-|Nokia|SonyEricsson|SIE-|Amoi|ZTE/.test(navigator.userAgent))){
    if(window.location.href.indexOf("?mobile")<0){
        try{
            if(/Android|Windows Phone|webOS|iPhone|iPod|BlackBerry/i.test(navigator.userAgent)){
                window.location.href="http://mob.xjtu.edu.cn/";
            }
```

10.3 给出 DNS 和 TCP 连接及获得 HTTP 完整应答的时间（单位 ms）。

DNS 时间	10.734 ms
TCP 连接建立的时间	4.819 ms
HTTP 的完整时间	14.004 ms

进阶设计

步骤 11：HTTP 客户端程序的改进，完整下载一个页面，即包括所有的内嵌对象的下载。给出代码。运行并记录结果，包括所有的 DNS 和 TCP 连接及获得 HTTP 完整应答的时间。

11.1 给出代码（文字）。

```
import socket
import sys
import time
from html.parser import HTMLParser
from urllib.parse import urlparse, urljoin

USER_AGENT = "Zhang Zhaoyuan"

class EmbeddedObjectParser(HTMLParser):
    def __init__(self):
        super().__init__()
        self.urls = []

    def handle_starttag(self, tag, attrs):
        attrs = dict(attrs)

        if tag == "img" and "src" in attrs:
            self.urls.append(attrs["src"])

        elif tag == "script" and "src" in attrs:
            self.urls.append(attrs["src"])

        elif tag == "iframe" and "src" in attrs:
            self.urls.append(attrs["src"])

        elif tag == "source" and "src" in attrs:
            self.urls.append(attrs["src"])

        elif tag == "link":
            rel = attrs.get("rel", "").lower()
            href = attrs.get("href")
            if href and "stylesheet" in rel:
                self.urls.append(href)

def recv_all(sock):
    data = b""
    while True:
        part = sock.recv(4096)
        if not part:
            break
        data += part
    return data

def parse_headers(header_bytes):
    lines = header_bytes.decode("iso-8859-1").split("\r\n")
    status_line = lines[0]
    headers = {}

    for line in lines[1:]:
        if ": " in line:
            k, v = line.split(": ", 1)
            headers[k.lower()] = v

    parts = status_line.split(" ", 2)
    status_code = int(parts[1]) if len(parts) >= 2 else 0
    return status_code, headers

def decode_chunked(body):
    result = b""
    i = 0
    while True:
        j = body.find(b"\r\n", i)
        size_line = body[i:j]
        size = int(size_line.decode().split(";", 1)[0], 16)
```

```
            if size == 0:
                break
            i = j + 2
            result += body[i:i + size]
            i += size + 2
    return result

def fetch_once(url):
    parsed = urlparse(url)
    host = parsed.hostname
    port = parsed.port or 80
    path = parsed.path if parsed.path else "/"
    if parsed.query:
        path += "?" + parsed.query

    dns_start = time.perf_counter()
    addr_info = socket.getaddrinfo(host, port, socket.AF_INET, socket.SOCK_STREAM)
    dns_end = time.perf_counter()

    ip = addr_info[0][4][0]

    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    tcp_start = time.perf_counter()
    sock.connect((ip, port))
    tcp_end = time.perf_counter()

    host_header = host if port == 80 else f"{host}:{port}"
    request = (
        f"GET {path} HTTP/1.1\r\n"
        f"Host: {host_header}\r\n"
        f"User-Agent: {USER_AGENT}\r\n"
        f"Accept-Encoding: identity\r\n"
        f"Connection: close\r\n"
        f"\r\n"
    )

    http_start = time.perf_counter()
    sock.sendall(request.encode())
    raw_response = recv_all(sock)
    http_end = time.perf_counter()

    sock.close()

    header_bytes, body = raw_response.split(b"\r\n\r\n", 1)
    status_code, headers = parse_headers(header_bytes)

    if headers.get("transfer-encoding", "").lower() == "chunked":
        body = decode_chunked(body)

    return {
        "url": url,
        "host": host,
        "ip": ip,
        "status": status_code,
        "headers": headers,
        "body": body,
        "dns_ms": (dns_end - dns_start) * 1000,
        "tcp_ms": (tcp_end - tcp_start) * 1000,
        "http_ms": (http_end - http_start) * 1000,
    }

def fetch_with_redirect(url, max_redirect=3):
    records = []
    current_url = url

    for _ in range(max_redirect + 1):
        result = fetch_once(current_url)
        records.append(result)

        if result["status"] in (301, 302, 303, 307, 308):
            location = result["headers"].get("location")
            if not location:
                break
            next_url = urljoin(current_url, location)
```

```
                if not next_url.startswith("http://"):
                    break
                current_url = next_url
            else:
                break

    return records

def unique_keep_order(urls):
    seen = set()
    result = []
    for u in urls:
        if u not in seen:
            seen.add(u)
            result.append(u)
    return result

def main():
    target = sys.argv[1]

    if not target.startswith("http://"):
        target = "http://" + target

    all_records = []

    # 下载主页面
    main_records = fetch_with_redirect(target)
    all_records.extend(main_records)
    final_main = main_records[-1]

    html = final_main["body"].decode(errors="ignore")

    # 解析内嵌对象
    parser = EmbeddedObjectParser()
    parser.feed(html)

    object_urls = []
    for u in parser.urls:
        abs_url = urljoin(final_main["url"], u)
        if abs_url.startswith("http://"):
            object_urls.append(abs_url)

    object_urls = unique_keep_order(object_urls)

    # 下载所有内嵌对象
    for obj_url in object_urls:
        obj_records = fetch_with_redirect(obj_url)
        all_records.extend(obj_records)

    # 输出结果
    print("===== Download Result =====")
    print(f"Main Page: {final_main['url']}")
    print(f"Embedded Object Count: {len(object_urls)}")
    print()

    total_dns = 0.0
    total_tcp = 0.0
    total_http = 0.0

    for i, r in enumerate(all_records, 1):
        total_dns += r["dns_ms"]
        total_tcp += r["tcp_ms"]
        total_http += r["http_ms"]

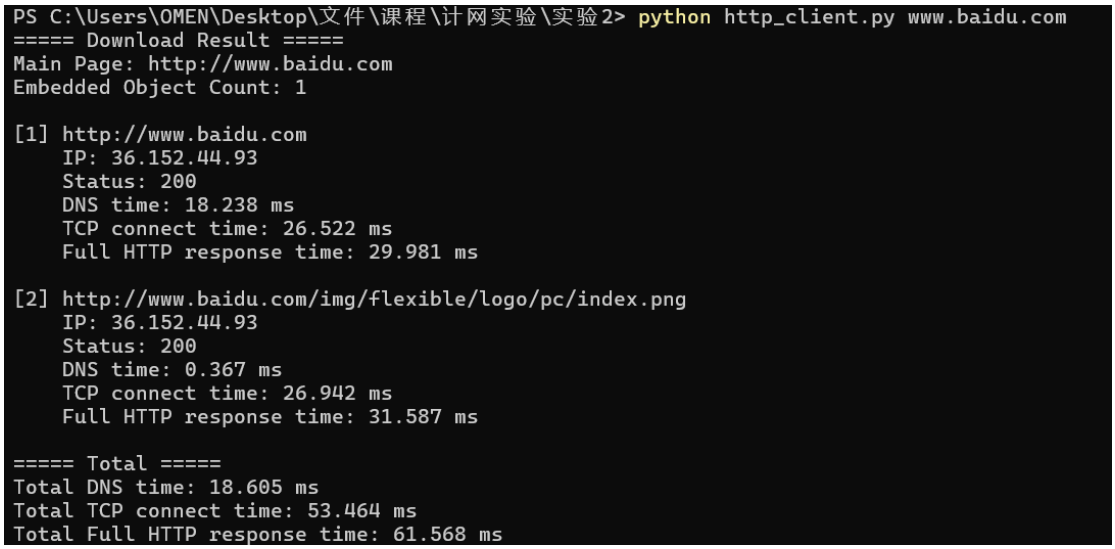
        print(f"[{i}] {r['url']}")
        print(f"    IP: {r['ip']}")
        print(f"    Status: {r['status']}")
        print(f"    DNS time: {r['dns_ms']:.3f} ms")
        print(f"    TCP connect time: {r['tcp_ms']:.3f} ms")
        print(f"    Full HTTP response time: {r['http_ms']:.3f} ms")
        print()

    print("===== Total =====")
    print(f"Total DNS time: {total_dns:.3f} ms")
    print(f"Total TCP connect time: {total_tcp:.3f} ms")
```

```
    print(f"Total Full HTTP response time: {total_http:.3f} ms")

if __name__ == "__main__":
    main()
```

11.2 给出测试结果（贴图）。



11.3 给出 DNS 和 TCP 连接及获得 HTTP 完整应答的时间（单位 ms）。

DNS 时间	18.605
使用了几个 TCP 连接	2
TCP 连接建立的时间	53.464
HTTP 的完整时间	61.568